

Arias_Group — Comprehensive Security & Code Audit Report

Audit Date: 2026-04-25
Repository: https://github.com/Oligonari2810/Arias_Group.git
Commit: a0bbbe1b1bbb0b81c20847f47998a644b52461fe
Auditor: Qwen Code (AI Assistant)
Scope: Full-stack security, code quality, dependencies, tests, infrastructure

Executive Summary

The Arias_Group application is a **Flask-based business operating system** for Fassa Bortolo/Gypsotech distribution in the Caribbean. The codebase demonstrates **strong security fundamentals** with several areas requiring immediate attention.

Overall Risk Assessment: MEDIUM

Category	Score	Status
Security	7.5/10	⚠ Needs Attention
Code Quality	8.0/10	✅ Good
Test Coverage	8.5/10	✅ Good (unit tests)
Dependencies	6.5/10	⚠ Vulnerabilities Found
Documentation	9.0/10	✅ Excellent
Infrastructure	8.0/10	✅ Good

1. Security Findings

1.1 Critical Issues (P0)

 VULNERABLE DEPENDENCIES (CVEs Detected)

Severity: HIGH

Location: requirements.txt

Name	Version	Vulnerability ID	Fix Versions
python-dotenv	1.2.1	GHSA-mf9w-mj56-hr94	1.2.2
pillow	11.3.0	GHSA-cfh3-3jmp-rvhc	12.1.1
pillow	11.3.0	GHSA-whj4-6x5x-4v2j	12.2.0

Impact: - python-dotenv : Potential environment variable leakage -
pillow : Remote code execution via malicious image files

Recommendation:

```
pip install --upgrade python-dotenv==1.2.2 pillow==12.2.0
```

1.2 High Priority Issues (P1)

 SESSION SECURITY CONFIGURATION

Location: app.py:78-82

```
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
app.config['SESSION_COOKIE_SECURE'] = not _debug #  Correct
```

Status:  **PROPERLY CONFIGURED** — Session cookies have appropriate security flags.

🟡 CSRF PROTECTION

Location: app.py:83 , templates/base.html

```
csrf = CSRFProtect(app)
```

Status:  **PROPERLY IMPLEMENTED** - Flask-WTF CSRF protection enabled globally - CSRF token injected via meta tag in base template - Auto-injection helper for fetch() requests

🟡 SQL INJECTION PROTECTION

Location: app.py:90-117 , db/adapter.py

```
def _safe_add_column(db: sqlite3.Connection, table: str, col: str, ty
    """ALTER TABLE seguro: valida identifiier y tipo contra allowlist."""
    if not _SAFE_IDENTIFIER_RE.match(table):
        raise ValueError(f"Identifiier inseguro de tabla: {table!r}")
    # ... allowlist validation
```

Status:  **PROPERLY PROTECTED** - Parameterized queries used throughout - Allowlist validation for DDL operations - SQL translation layer for PostgreSQL compatibility

🟡 OPEN REDIRECT PROTECTION

Location: app.py:86-95

```
def _safe_next_url(target: str | None) -> str | None:
    """Permite solo paths internos relativos. Bloquea open redirect a
    if not target:
        return None
    parsed = urlparse(target)
    if parsed.scheme or parsed.netloc:
        return None
    return target if target.startswith('/') and not target.startswith(
```

Status:  **PROPERLY PROTECTED** — Login redirect validation implemented correctly.

PASSWORD HANDLING

Location: `app.py:1951-1964`

```
password = request.form.get('password', '')
# ...
if user and bcrypt.checkpw(password.encode(), user['password_hash'].en
```

Status:  **PROPERLY IMPLEMENTED** - bcrypt password hashing (industry standard) - Passwords loaded from environment variables (`.env.example`) - No hardcoded credentials found

1.3 Medium Priority Issues (P2)

MISSING RATE LIMITING

Location: `/login` endpoint

Severity: MEDIUM

CWE: CWE-307 (Improper Restriction of Authentication Attempts)

Finding: No brute-force protection on login endpoint.

Recommendation:

```
# Add flask-limiter
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(app, key_func=get_remote_address)

@app.route('/login', methods=['GET', 'POST'])
@limiter.limit("5 per minute")
def login():
    # ...
```

🟡 POTENTIAL IDOR VULNERABILITY

Location: PDF export endpoints (/api/offer-pdf/<id>)

Severity: MEDIUM

CWE: CWE-639 (Authorization Bypass Through User-Controlled Key)

Finding: Endpoints may not validate ownership of resources.

Recommendation: Add ownership validation:

```
@app.route('/api/offer-pdf/<int:offer_id>')
@login_required
def offer_pdf(offer_id):
    db = get_db()
    offer = db.execute('SELECT * FROM pending_offers WHERE id = ?', (offer_id,)).fetchone()
    if not offer:
        abort(404)
    # Add ownership check if multi-user:
    # if current_user.role != 'admin' and offer['created_by'] != current_user.id:
    #     abort(403)
```

🟡 HARDCODED FILE PATHS

Location: backup_db.sh

```
BASE_DIR="/Users/anamarperezmarrero/Mvp_Arias_Fassa"
```

Severity: LOW-MEDIUM

Recommendation: Use environment variables:

```
BASE_DIR="${FASSA_BASE_DIR:-$(pwd)}"
```

1.4 Security Strengths

✔ **Well-Implemented Security Controls:**

- 1. **CSRF Protection** — Flask-WTF with global CSRF + JS auto-injection
- 2. **Session Hardening** — HTTPOnly, SameSite=Lax, Secure (prod-only)
- 3. **SQL Injection Prevention** — Parameterized queries, allowlist validation
- 4. **Password Security** — bcrypt hashing, env-based seed passwords
- 5. **Open Redirect Prevention** — `_safe_next_url()` helper
- 6. **XSS Prevention** — Jinja2 auto-escaping (default in Flask)
- 7. **Database Adapter Security** — SQL translation layer with PRAGMA blocking

2. Code Quality Assessment

2.1 Architecture

Pattern: Flask monolith with emerging modular structure

```
app.py (4969 lines) – Main application
├─ Database layer (db/) – PostgreSQL adapter skeleton
├─ Logistics engine (logistics/engine.py) – Pure Python calculation
├─ Exports (exports/) – PDF/Excel generation
└─ Templates (templates/) – 15 HTML templates
```

Assessment: ✔ **GOOD** — Clear separation of concerns emerging.

2.2 Code Metrics

Metric	Value	Assessment
Total Lines (app.py)	4,969	⚠ Large file
Total Routes	~50	⚠ Many endpoints
Functions (calc engine)	8	✔ Well-tested
Test Coverage (unit)	100%	✔ Excellent

Metric	Value	Assessment
Test Coverage (app.py)	15.8%	⚠️ Low (expected for unit-only)

2.3 Code Style & Conventions

Strengths: - ✅ Consistent naming conventions (Spanish domain language) -
✅ Type hints on critical functions - ✅ Comprehensive docstrings on complex logic -
✅ Dataclasses for structured data (logistics)

Areas for Improvement: - ⚠️ Function length: Some functions exceed 100 lines -
⚠️ Magic numbers: Some hardcoded values (e.g., discount percentages) -
⚠️ Comment density: Could benefit from more "why" comments

2.4 Critical Functions Review

`calculate_quote()` (lines 1859-1945)

Purpose: Top-level quote orchestration

Assessment: ✅ **WELL-STRUCTURED** - Clear input/output contract - Proper error handling - Comprehensive test coverage (90%+)

`compute_logistics()` (logistics/engine.py)

Purpose: Container optimization engine

Assessment: ✅ **EXCELLENT** - Pure function (no side effects) - Well-documented business logic - Comprehensive unit tests

`init_db()` (app.py:402-625)

Purpose: Database initialization + migrations

Assessment: ⚠️ **NEEDS REFACTORING** - 223 lines in single function - Multiple migration functions called sequentially - Consider extracting to `db/migrations.py`

3. Test Coverage Analysis

3.1 Test Suite Status

```
tests/
├─ unit/                                # 101 tests – ALL PASSING ✅
│   ├── test_num.py
│   ├── test_detect_family.py
│   ├── test_compute_line.py
│   ├── test_container_result.py
│   ├── test_estimate_containers.py
│   ├── test_compute_totals.py
│   ├── test_dedup_alerts.py
│   └─ test_logistics_engine.py
├─ integration/
│   ├── test_calculate_quote.py
│   ├── test_db_skeleton.py
│   └─ test_offer_pipeline.py
└─ test_coverage_gate.py                # Per-function coverage validation
```

3.2 Coverage Results

Component	Coverage	Target	Status
Calculation Engine	100%	≥85%	✅ PASS
calculate_quote()	92%	≥90%	✅ PASS
app.py (overall)	15.8%	N/A	⚠️ Low (unit tests only)

3.3 Test Quality

Strengths: - ✅ Boundary condition testing - ✅ Regression tests for known bugs - ✅ Property-based assertions - ✅ Clear test naming

Gaps: - ⚠️ Limited integration tests (PostgreSQL) - ⚠️ No end-to-end UI tests - ⚠️ Missing security tests (auth bypass, rate limiting)

4. Dependency Audit

4.1 Production Dependencies

Package	Version	Status	Notes
Flask	3.1.3	✓ Current	Latest stable
Flask-Login	0.6.3	✓ Current	
Flask-WTF	1.3.0	✓ Current	
bcrypt	5.0.0	✓ Current	
ReportLab	4.4.10	✓ Current	
openpyxl	3.1.5	✓ Current	
SQLAlchemy	2.0.49	✓ Current	
Alembic	1.18.4	✓ Current	
psycopg	3.3.3	✓ Current	
python-dotenv	1.2.1	● VULNERABLE	Upgrade to 1.2.2
Pillow	11.3.0	● VULNERABLE	Upgrade to 12.2.0

4.2 Development Dependencies

Package	Version	Status
pytest	9.0.3	✓ Current
pytest-cov	7.1.0	✓ Current
coverage	7.13.5	✓ Current

5. Database Architecture

5.1 Current State

Primary: SQLite (`fassa_ops.db`)

Target: PostgreSQL (skeleton in `db/`)

Schema: - 19 tables (clients, products, projects, offers, etc.) - Proper indexing - Foreign key constraints - Audit logging

5.2 Migration Status

Alembic migrations:  **CONFIGURED** - `0001_initial_schema.py` — Full PostgreSQL schema - Enum types for status fields - Proper type mappings (NUMERIC, TIMESTAMPTZ, JSONB)

Assessment:  **WELL-PREPARED** for PostgreSQL migration.

5.3 Data Integrity

Strengths: -  Parameterized queries throughout -  Transaction management -  Audit log table -  Price history tracking

Recommendations: -  Add database-level constraints (CHECK constraints) -  Consider row-level security for multi-tenant scenarios

6. Infrastructure & DevOps

6.1 Docker Configuration

File: `docker-compose.yml`

```
services:
  postgres:          # Port 5434, persistent
  postgres-test:     # Port 5433, tmpfs (volatile)
```

Assessment:  **WELL-CONFIGURED** - Separate dev/test databases - Health checks - Volume management

6.2 CI/CD (GitHub Actions)

File: `.github/workflows/tests.yml`

Assessment:  **GOOD** - Runs on push + PR - Python 3.11 - Coverage gate enforcement - Artifact upload

Recommendations: -  Add security scanning (pip-audit, bandit) -  Add linting (ruff, mypy) -  Consider matrix testing (Python 3.10, 3.11, 3.12)

6.3 Environment Configuration

File: `.env.example`

Assessment:  **COMPREHENSIVE** - SECRET_KEY generation documented - DATABASE_URL for PostgreSQL - Seed password configuration - Clear separation of concerns

7. Documentation Quality

7.1 Documentation Inventory

Document	Status	Quality
README.md	 Complete	Excellent
HANDOFF.md	 Complete	Excellent
SPEC-001.md	 Complete	Excellent
docs/design/	 Present	Good
docs/deployment/	 Present	Good

7.2 Documentation Strengths

 **README.md:** - Clear setup instructions - Architecture diagram - Module overview table - Quick start guide

✅ **HANDOFF.md**: - ERP migration roadmap - Data model mapping (Odoo-compatible) - JSON export contract - Gap analysis

✅ **SPEC-001.md**: - Clear acceptance criteria - Test structure specification - Coverage requirements

8. Recommendations

8.1 Immediate Actions (P0 — This Week)

1. **Upgrade Vulnerable Dependencies:**

```
bash pip install --upgrade python-dotenv==1.2.2  
pillow==12.2.0
```

2. **Add Rate Limiting to Login:**

```
python pip install flask-limiter # Add  
@limiter.limit("5 per minute") to /login
```

3. **Fix Hardcoded Paths:**

4. Update `backup_db.sh` to use environment variables

8.2 Short-Term Actions (P1 — This Month)

1. **Add IDOR Protection:**

2. Review all `/api/<resource>/<id>` endpoints

3. Add ownership validation

4. **Expand Test Coverage:**

5. Integration tests for PostgreSQL

6. Security tests (auth, CSRF bypass attempts)

7. End-to-end tests for critical flows

8. **Add CI Security Scanning:** ```yaml

9. name: Security audit run: pip-audit -r requirements.txt

10. name: Linting run: ruff check . ```

8.3 Medium-Term Actions (P2 — This Quarter)

1. Refactor Large Functions:

2. Split `init_db()` into smaller migration functions

3. Extract route handlers to blueprints

4. PostgreSQL Migration:

5. Complete SPEC-002b (schema migrations)

6. Test with production data volume

7. Add Monitoring:

8. Health check endpoint

9. Performance metrics

10. Error tracking (Sentry integration)

8.4 Long-Term Actions (P3 — This Year)

1. ERP Integration Readiness:

- Implement JSON export endpoint (§4 HANDOFF.md)
- Rename legacy fields to Odoo-compatible names
- Add factory/logistics order auto-generation

2. Scalability Improvements:

- Connection pooling
 - Caching layer (Redis)
 - Background job processing (Celery)
-

9. Compliance Considerations

9.1 Data Protection

GDPR/LPDP Considerations: -  Personal data (clients) stored with RNC (tax ID) -  No data retention policy implemented -  No data export/deletion endpoints

Recommendations: - Add data retention configuration - Implement right-to-erasure endpoint - Document data processing activities

9.2 Audit Trail

Current State:  **GOOD** - `audit_log` table tracks offer changes - `price_history` tracks price changes - `stage_events` tracks project progression

Enhancement: - Add user login tracking - Export audit logs for compliance reporting

10. Conclusion

10.1 Overall Assessment

The Arias_Group application demonstrates **strong engineering fundamentals** with a solid security foundation. The codebase is well-documented, tested (for critical calculation logic), and architected for future growth.

Key Strengths: -  Security-first mindset (CSRF, SQL injection, open redirect protection) -  Comprehensive documentation -  Strong test coverage on business-critical logic -  PostgreSQL migration prepared

Key Risks: -  Vulnerable dependencies (immediate fix required) -  Missing rate limiting (brute-force vulnerability) -  Low overall test coverage (15.8% on app.py)

10.2 Risk Score Summary

Category	Risk Level	Priority
Dependencies	 HIGH	P0
Authentication	 MEDIUM	P1
Authorization	 MEDIUM	P1

Category	Risk Level	Priority
Data Protection	● LOW	P2
Infrastructure	● LOW	P2

10.3 Next Steps

1. **Immediate:** Fix vulnerable dependencies (python-dotenv, pillow)
2. **This Week:** Add rate limiting to `/login`
3. **This Month:** Expand test coverage, add security scanning to CI
4. **This Quarter:** Complete PostgreSQL migration, refactor large functions

Audit Completed: 2026-04-25

Next Scheduled Audit: 2026-07-25 (Quarterly)

This audit was conducted using automated tools (pip-audit, pytest) and manual code review. Findings should be validated before production deployment.